Graphs

Graph Algorithms

Quick Reference

Graph refers to any arbitrary graph. *DiGraph* refers specifically to directed graphs. There are 12 available black boxes.

By the way, all algorithms that have a logarithmic runtime are log n. This is because of a trick that lets us simplify log m to log n^2 = 2 log n = O(log n). Don't even bother with this; just remember that nothing is log m, everything is log n.

Name	**Inputs**	**Outputs**	**Runtime**
DFS	Graph, source	ccnum[], prev[], pre[], post[]	O(n+m)
BFS	Graph, source	dist[], prev[]	O(n+m)
Topological sort	DAG	order[]	O(n+m)
SCC	DiGraph	G_SCC (metagraph)	O(n+m)
Dijkstra's	Graph, source, weights (positive)	dist[], prev[]	O((n+m) log n)
Bellman-Ford	Graph, source, weights	dist[], prev[], iter[][]	O(nm)
Floyd-Warshall	Graph, weights	dist[][], iter[][][]	O(n^3)
Kruskal's	UndirectedGraph, weights	edges[]	O(m log n)
Prim's	Undirected Graph, weights	prev[]	O(m log n)
Ford-Fulkerson	DiGraph, capacities (integer), source, sink	flow[], C (maxflow)	O(mC)
Edmonds-Karp	DiGraph, capacities, source, sink	flow[], C (maxflow)	O(nm^2)
2-SAT	CNF	assignments[], SCC outputs	O(n+m)

I like to categorise them as such:

Shortest Path Algorithms	DFS (and applications)	MSTs	Flow
BFS	DFS	Prim's	Ford-Fulkerson
Dijkstra's	Topological Sort	Kruskal's	Edmonds-Karp
Bellman-Ford	SCC		
Floyd-Warshall	2-SAT		

Shortest Path Algorithms

What is the shortest/fastest/most optimal path?

All of these algorithms contend with shortest paths. BFS, Dijkstra's and Bellman-Ford are all single-source shortest path algorithms, and therefore **require a source**. Floyd-Warshall is the only all-source shortest path algorithm, so a source is not provided.

Select the appropriate shortest path algorithm by the following heuristic:

1. Use BFS whenever possible; O(n+m)
2. Unless of course the edges are weighted, then use Dijkstra's; O((n+m) log n)
3. Unless of course the edges are *negatively weighted*, then use Bellman-Ford; O(nm)
4. Unless of course all sources are required, then use Floyd-Warshall; O(n^3)

If you find yourself running BFS/Dijkstra's on all vertices, ABORT and reconsider the plan.

All of these algorithms can:

- Test for connectivity if given source `u`, `dist[v] != inf`. Floyd-Warshall is a bit special, check if `dist[u][v] != inf` instead. Generally, test for connectivity with BFS as it is the fastest.
- Work on reversed graphs to efficiently determine pathing *to* a vertex instead of *from* a vertex. Of course use BFS/Dijkstra's as the frontliners.
- Find the distance from a source to all nodes from one pass of BFS/Dijkstra's (via the `dist` array)
    - In fact, if a directed graph is reversed, then the distance from all nodes to some target can be found. This seems to be a common trick in the practice problems.

BFS Features

1. Mostly used for unweighted graphs
    - Can run on weighted graphs, though this will just count the number of edges while ignoring edge weights.
2. Can be used to determine what happens if a specific edge (or certain edges are) removed from the graph.

Dijkstra's Features

1. A single negative weight will ruin it. Do not attempt to run Dijkstra's on a graph with potentially negative weights.
2. Also note that simply transforming the graph to only containing positive weights (e.g., by adding some large constant) does not work.
    - Consider as an example two paths from s to t. The first path contains three edges, all with weight -1. The second path contains one edge directly from s to t with weight -2. It should be clear that the first path is shorter (-3 in total).
    - If we add +2 to all weights, then the first path will have three edges with weight 1. The second path will have weight 0. Now the second path appears to be shorter.

    - Basically, adding weights compounds over the number of edges in a particular path.
3. Can be used to determine what happens if a specific edge is (or certain edges are) removed from the graph.

Bellman-Ford Features

1. Mainly a version of single-source shortest path that allows *negative edges,* though there is a runtime cost.
    - If there is only one negative edge (or some constant number), consider modifying the graph to run Dijkstra's as it is faster.
2. Bellman-Ford can detect negative cycles but only from the given source.
3. Bellman-Ford is quite simple. The trick is to ensure that there are *absolutely* no other faster alternatives.

Floyd-Warshall Features

1. Floyd-Warshall's gimmick is being all-source instead of single-source like the rest of the shortest path algorithms
2. Floyd-Warshall is quite simple. The trick is to ensure that there are *absolutely* no other faster alternatives.
3. Floyd-Warshall can detect negative cycles anywhere in the graph. Positive cycles are trickier (just ignore this).

DFS (and applications)

Is it connected?

All of these algorithms use DFS as the underlying strategy. Therefore, all of these algorithms run in O(n+m) time. Consider how impressive that is, then commit to memory.

DFS Features

1. Can be used to check for connectivity (look at `ccnum`), but stick with BFS as it is easier.
2. Consider using DFS's pre and post numbers whenever terms reminiscent of relatives appear (ancestor, descendant)
3. DFS can technically be used to check for cycles (if there is a back-edge) but this use case feels extremely niche.

SCC Features

1. Only works on directed graphs -- there is no such thing as strongly connected in an undirected graph.
2. If a metagraph of SCCs is required, then use the SCC algorithm. It is likely that further processing of this metagraph is required.
3. SCC can be used to detect cycles quite quickly if the number of meta-vertices is less than the number of vertices in the original graph. Note some quirks (see below)
4. Quirks:
    a. Do not give details about the SCC algorithm unless absolutely necessary. This will cost points if mistakes are made.
    b. A SCC need not contain a cycle; a single node *is* a strongly connected component. An SCC can also contain multiple cycles.

Topological Sort Features

1. Sorting a DAG gives an ordering from source to sink. Note that an SCC metagraph gives a reverse ordering of sink to source.
2. Pro-tips:
    a. There is no need to provide details on finding the first source/sink, as these will always exist.
    b. However, there is a need to provide details on finding other sources and other sinks. This can be done by iterating through the adjacency list and finding vertices with no neighbours.
    c. Simply state that the vertices are processed "from source to sink" or "from sink to source" and it will be accepted.

2-SAT Features

1. While other problems involve reducing to a Graph, 2SAT first requires the problem to be reduced to a 2-satisfiability CNF.
2. Note that there is no need to reduce further into a graph, since 2SAT handles this automatically.

MSTs

Notes

MSTs are generally applied on **connected, undirected** graphs.

For black-box purposes, I think it's safe to just stick with Kruskal's, but make sure the differences are well understood! Both algorithms run in O(m log n) time.

- Kruskal's explicitly returns the edges that are part of the MST, which is much more helpful. Prim's returns some prev array which sounds like a pain in the ass.
- Kruskal's sorts edges by weight and adds the lightest edge that does not create a cycle. Do this until all edges that will not create a cycle are added (m = n-1 ; by the property of a tree).
- Prim's starts with an arbitrary vertex v in S and grows S by adding the lightest edge between a vertex in S and a vertex outside of S (cut property). Do this until all vertices are in S.

Both MST algorithms are greedy algorithms based on the **cut property**. You can just name-drop the cut property if it helps. Prim's algorithm demonstrates the cut property much more explicitly.

Suppose edges X are part of an MST of a graph. Pick any subset of nodes S for which X does not cross between S and V - S, and let e be the lightest edge across this partition. Then X + e is part of some MST.

I find it helps to think of the "cut" as an arbitrary division that does not slice through any existing MST edges. This means that the MST-in-progress can be disjoint; that's perfectly fine.

Optionally, know the **cycle property**. Same thing, just name-drop with the brief explanation below.

In a cycle, the heaviest edge cannot be part of an MST.

Properties

Algorithms

- The MST algorithms do not care about negative weights

Edge weights

- The edge of minimum weight will always be in the MST
- The edge of maximum weight might be in the MST (if it is the only thing connecting a single vertex)
- The edge of maximum weight in a cycle will never be part of an MST.
- The edge of minimum weight in a cycle might be in the MST (if there are two conjoined cycles and the minimum weight edge is a maximum weight edge in a different cycle)

- Modifying all edge weights by some constant factor **does not** change the MST.
- If all edge weights are distinct then there is a unique MST.

Cycles and cuts

- Adding an edge to an MST creates a cycle
- Removing an edge from an MST creates two disconnected components

Miscellaneous

- A shortest path might not necessarily be part of some MST. A path does not necessarily traverse all vertices after all.
- The number of edges in an MST is O(n), not O(m)

Strategies

- Remove an edge, or certain edges and run Kruskal's or Prim's to determine what happens without the edge(s)
- Make changes to the weights to force/avoid edges being picked first
    - Note that negatives are allowed here. Unlike shortest path problems, shifting all edge weights for MST problems is okay -- we are handling one edge at a time.
    - Think carefully about the weight changes. Negative infinity will force an edge to be picked first. Positive infinity will force an edge to be picked last. Weights do not stop an edge from being picked (just delete them).
- Run Kruskal's or Prim's on a subset of G (if some requirements are needed), then build on the partial MST.
- Use the MST properties (cut, cycle, n-1, etc.) to justify correctness.

Flow

Notes

The flow algorithms take in a lot of inputs:

a) The directed graph
b) The source s (infinite capacity)
c) The sink t (infinite capacity)
d) The capacities c

For using black-boxes, just summarise all of these inputs as the "flow network". Of course take the time to define each input, but "flow network" refers to all of these together.

- Notice that flow is not a provided input. Therefore, the flow algorithms cannot start at a partial flow. It is possible to encode the flow by subtracting the existing flow by the edge capacities and sending that new graph into a black-box algorithm.

Denote the residual flow network G^f.

Properties

- A min-cut is the minimum capacity across an s-t cut, not the minimum flow.
- A bottleneck edge is an edge where increasing its capacity will increase the max-flow.
- A critical edge is an edge where decreasing its capacity will decrease the max-flow.
- All bottleneck edges are => critical edges. All critical edges are => saturated edges. The reverse implication is not true.
- All min-cut edges are => saturated edges. The reverse implication is not true.
- If the size of some flow = the capacity of some cut (L, R), then the given flow is always the max-flow.
    - Then the given cut is a min-cut
    - Then for every other valid flow, the capacity of the cut (L, R) >= the size of the flow.
    - Then every other cut >= the size of the given flow.

Strategies

- Turning a graph into a flow network might solve the problem (e.g., bipartite graph with capacities 1); see below under Notes to self > Common Techniques > Problems > Flow.
- Nodes or edges can be added to divert or add flow.
- Find a max flow, then create a residual network and work with it as a graph with BFS/DFS.
- A common trick is to add super-sources or super-sinks.
- If the input is modified and the answer asks for a max flow, then tidy up the output (e.g., remove all super-sources and super-sinks) before returning it.
- Flow algorithms give a flow array that has the flow value for each edge for easy access.

Exam Info

Answer Format

Algorithm Checklist:

1. No pseudocode
2. Explicitly state the name of the black box(es) used. Simply saying "graph traversal" is not enough, and will be penalised.
3. Do not modify the black box.
4. Only the input/output can be modified. Detail any such modifications if needed.
5. Detail any data or return extractions if needed. Clean up return values if necessary (e.g., if edges were previously modified or vertices were previously added)
6. As with Divide and Conquer, too much detail is better than too little.
7. Bullets are allowed but not preferred. If possible, use prose.
    a. Sub-bullets are discouraged.

Justification of Correctness Checklist:

1. Why was the black box chosen?
2. Why were modifications to the input/output made?
    a. If converting to a graph, simply state "by modelling the problem as a graph, we are able to efficiently solve the problem using the properties of <black box>..."
3. Why does the returned value(s) solve the problem?

Runtime Analysis

- Since the black boxes are not modified, simply assert the run time of these known algorithms.
- However, if any input/output modifications were performed, these need to be stated.
    - This includes converting the original problem into a graph.
- Same thing goes for return extractions -- state the runtime.
- Runtimes should be in terms of n and/or m . We assume that n = number of vertices and m = number of edges.
    - In general, try to use both terms. It is difficult to make any assumptions about the graph.
    - In some cases (e.g., minimum spanning tree or max flow), we might know that the graph is connected. Therefore, it makes sense here that m >= n , so we can drop n since m, the number of edges, dominates.
    - When in doubt it is probably safer to default to using both n and m .

Problem Format

Some problems are not given as graphs, so there is a need to frame the problem as a graph to apply the unmodified black boxes provided.

Alternatively, some graphs might have to be modified such that an unmodified black box can solve the problem.

Then, all that's left to do is to select the correct black box based on the input/output needs, though this is often quite difficult as well (more below).

In short, this subject is a form of **reduction**.

Common Mistakes

Algorithm

Shortest Path Algorithms and DFS (and applications)

- Checks for a single source/sink, but not others. Note that a single vertex is both a source and a sink (a source/sink is defined by the number of incoming/outgoing edges).
- Assumes that there is a single source/sink.
- Solves the problem for connectivity to a single v instead of between all pairs of vertices.
- Does not provide all inputs to a blackbox, especially the source vertex for algorithms like BFS/Dijkstra's.
- Checks connectivity with DFS without additional details.
    - The `visited` array in DFS will be useless at the end of the run (everything will be set to true).
    - Checking the visited array during the run is not allowed as that counts as black-box modification.
    - BFS is a much more natural way to check for connectivity.

MST

- Runs an MST algorithm from scratch (if an MST is already provided).
- Algorithm results in an invalid MST.
- Algorithm does not solve the problem requirements.
- Does not provide all inputs to a black box.
- Checks connectivity with DFS without additional details. Use BFS; makes it easier. You can assume that MSTs are on connected graphs unless otherwise stated.

Justification of Correctness

Shortest Path Algorithms and DFS (and applications)

- Does not explain how the output solves the problem
- Does not explain why we check for every source/sink
- Does not explain why we need the SCC metagraph
- Does not explain why some black-box was run multiple times.

MST

- Does not explain why edges are removed, added, or modified.

- Does not explain why an MST is still valid or possible after input modifications
- Does not explain how the output solves the problem.

Runtime Analysis

- Provides analysis but with an incorrect overall runtime.
- Simplifies runtime when the graph has unknown density. However, even if the graph has constraints, just report the black-box runtime. It will always be acceptable as correct.

***

Notes to Self

Common Techniques

Utility

Looping through all edges in a graph (find/remove/edit)

- Takes O(n+m) time given the adjacency list format (for all vertices, for all edges)

Looping through all edge weights/capacities/properties in a graph

- This takes O(m) time; edge weights are stored in a separate data structure that allows for magic indexing, no idea why this course handles things like this.

Reversing edges of a graph

- The details of this do not to be mentioned, just say "reverse the graph". This takes O(n+m) time; same as looping through all edges in the graph.
- First initialise an empty graph in adjacency list form for all n vertices. Then, for each vertex v in G, iterate through the neighbours u of v. For each of the neighbours u, add v as a neighbour of u in the reverse graph.

Finding the degree of a vertex

- Check the length of the adjacency list for each vertex. This takes O(n) time.

Finding the path from u to v using the prev array

- Just state exactly that, "using the prev array, find the path from u to v".

Finding sinks/sources in linear time

- To find sinks, simply scan the adjacency list. Vertices with no neighbours are sinks. This takes O(n) time -- there is no need to check the edges, just ask if this current vertex has neighbours or not.
- To find sources, reverse the edges of the original graph. Then, do the same scanning and checking for neighbours. The total runtime is O(n+m) + O(n) = O(n+m)

Building a residual network

- Just say “build a residual network”

Paths

Finding the longest path in a DAG (NP-Hard in undirected, a cyclic graph doesn't make sense)

- Topologically sort the DAG. Set all vertices to have a count of 1. Start from the source and for each child, increment the vertex by 1. For each other vertex, if count(vertex) > count(child), then set count(child) to count(vertex) + 1. Return the largest count.

Finding if a vertex disjoint path that touches all vertices exists in a directed graph

- Topologically sort the graph. From source to sink, check if a vertex has an edge to the vertex immediately following it in the sorted order. If an edge is missing in this check, return false otherwise return true at the end of the check.

Finding the shortest path after modifying one edge from a choice of E edges

- Run Dijkstra/BFS starting from s on the graph. This gives the shortest path from s to all other vertices (call this `SP(s, u)`).
- Then run Dijkstra/BFS starting from t on the reverse graph. This gives the shortest path from all other vertices to t (call this `SP(v, t)`).
- For each edge `e (u, v)` in E, the shortest path including this modification will be `SP(s, u) + e (u, v) + SP(v, t)`.
- Note that the final iteration cannot be ignored in terms of runtime.

Finding the shortest path where the weights are multiplicative

- Take the logarithm of all weights. Then run a regular shortest path algorithm. If a negative logarithm is taken (maximising instead), then we have to run Bellman-Ford.

Cycles

Finding if an edge e (u, v) causes a cycle

- Look at the graph without e (either remove it or leave it out for now). Check if there is a path from u to v (undirected), or v to u (directed). If there is, e will cause a cycle.

Finding a cycle caused by edge e (u, v)

- Look at the graph without e. Trace the path from u to v (undirected) or v to u (directed). Note that there might be multiple paths if u, v were previously already part of a cycle.

Finding the shortest cycle that includes edge e (u, v) in an undirected graph

- Use the appropriate shortest-path algorithm depending on the graph type.
- Either way, remove edge e and find the shortest path from u to v. The shortest cycle is therefore the length of this shortest path + edge e.

Finding the shortest cycle in a directed graph

- Run Floyd-Warshall. Keep track of the minimum length cycle as defined by `dist[u][v] + dist[v][u]` . If this length is infinite then no cycle actually exists.

Finding if a cycle exists in an undirected graph

- Run DFS. If a back-edge exists then a cycle exists. Concretely, for each directed edge u, v check if `post[u] < post[v]`.

Finding if a cycle exists in a directed graph

- Run the SCC algorithm. Check if the number of meta-vertices < number of vertices. If yes then a cycle exists.

Finding if a negative cycle exists in a directed graph

- Run Bellman-Ford. Compare `iter[n][i]` and `iter[n-1][i]` for all vertices `i`. If there is any
- vertex where the `n`-th iteration is smaller than the `n-1`-th iteration, a negative cycle exists. Note that the negative cycle is only detectable from the given source.
- Run Floyd-Warshall. If `dist[i][i]` is negative then there is a negative cycle for `i`.

MSTs

Finding a maximum spanning tree

- Flip all edge weights to negative (multiply by -1). Find the MST on that.

Finding the minimum C where C is the upper limit of all weights in a path from s to t

- This seems like a pathing problem, but can be much more efficiently solved with MST. Just run an MST and path from s to t on the MST. Return the maximum weight along that path.
- Note that we don't actually care about the overall distance from s to t.

Given an MST, modify an edge and find the new MST

- If the edge is not within the MST and is updated to be lighter, add the edge into the MST. This creates a cycle. Remove the heaviest edge within this cycle.
- If the edge is within the MST and is updated to be heavier, remove the edge from the MST. This creates two components. Add the lightest edge within this cut.

Finding an MST that prioritises certain edges A over edges Z

- For all edges Z, add the maximum weight of all edges A to its own weight. Run MST.

Find a spanning tree (not MST) that has vertices in U as leaves

- Delete all vertices in U from the graph. Create a partial MST from the resulting graph. Add U back to this MST by choosing the lightest edge that connects it to the partial MST, taking care to never connect it to another vertex that was in U.

Flow

Finding a min st-cut

- Run a flow algorithm on the flow network. Take the residual graph and run BFS from s. All vertices reachable from s fall on the s-side of the cut. All vertices not reachable are on the t-side of the cut.
- An extension is to find edges that cross the min-cut. Just iterate through all edges and find edges (u, v) where u and v are on two different sides of the cut.

Finding a critical edge (an edge that decreases max-flow if its capacity is reduced)

- Run Edmonds-Karp on the flow network. For each saturated edge (flow = capacity), run BFS from u on the residual graph to see if v is reachable.
- If it is reachable, then some flow can still travel from u to v by another means. It is therefore not critical. If it is not reachable, then it is critical.

Modifying one edge e (u, v) after max-flow has already been found

- First find a path from s to u and a path from v to t. This is the path that must also be corrected once we correct our edge (u, v).
- Modify the provided edge capacity by some amount X. Decrease the flow along the provided edge accordingly. Decrease the flow along s to u and v to t by that same amount X (rollback X flow).
- Set the capacities of the graph to be capacity - existing flow. This encodes the existing flow within the capacities, but allows us to use black-box algorithms which do not allow for partial flows. Run the black-box algorithm Ford-Fulkerson.
- Add the flow from the latest run back to the original flow of the corrected graph.

Reduce from multiple sources/sinks to the max-flow problem

- Connect a super-source to all sources and a super-sink to all sinks. The capacities of all such edges are set to infinite. Run Edmonds-Karp on the flow network. Tidy up the output if necessary.
- Infinite capacities do not matter since BFS/DFS does not care about weights/capacities.

Reduce from vertices with capacities to the max-flow problem

- Split each vertex v into two: v_in and v_out.
- All edges pointing to the original vertex v should be connected to v_in. All edges pointing out of the original vertex v should be similarly outgoing from v_out.
- Connect a single edge from v_in to v_out with the capacity of v.
- Run Edmonds-Karp on the flow network. Tidy up the output if necessary.

Check if a provided flow is a max-flow

- Verify that edge capacity constraints are met.
- Verify that conservation of flow is held.
- Build the residual graph G^f with the provided flow.
- From s, run BFS on the residual graph. Check if t is reachable from s (`dist[t] != inf`). If t is reachable then the provided flow is not a max-flow.

Maximum bipartite matching

- Add a super-source and a super-sink. All edges are set to have capacity 1. Run Edmonds-Karp on the flow network. The max-flow is the size of the maximum matching.

Maximum number of paths from s to t

- If edge disjoint (no edge used more than once), make all edges have capacity 1. Run Edmonds-Karp on the flow network.
- If vertex disjoint (no vertex used more than once), make all edges and vertices have capacity 1. Refer to *reduce from vertices with capacities to the max-flow problem* to solve. Max-flow = max number of paths.

Hazard Warning

1. Edge "weights" and edge "lengths" seem to be used interchangeably in the book. Take care to read and double-read the question.
2. Do not assume that the problem is given in graph format even if it just looks like a graph. Dedicate a few lines to converting the input into a graph. Note that this affects the justification of correctness and runtime analysis as well!
3. Note that a directed graph makes no guarantees about sources and sinks (unless it is a DAG). A path cycle has no sources and sinks.
4. A cycle is defined as a path from one vertex back to itself without using the same edge twice. A cycle must therefore be at least three vertices, since this course does not deal with anti-parallel edges.
5. Modeling a problem as longest-path is stupid -- this is NP-hard. It might be easier to make the weights negative and pay the Bellman-Ford cost O(nm) to solve a shortest-path problem instead.
6. Running any algorithm from every edge/vertex is a bad idea.